CONTRIBUTOR GUIDE

Contributing to PowerGlove Vision

Source style, testing, documentation, packaging, and pull-request expectations.

2026 EDITION / IAIN BENNETT / MIT LICENSE

Thank you for helping improve PowerGlove Vision. Keep changes focused, readable, and safe for a project that combines camera input, local networking, virtual Linux devices, and a privileged shutdown helper.

Before changing code

1. Start from the current `main` branch and create a short-lived topic branch.
2. Read `SECURITY.md` before changing pairing, tokens, network listeners, downloads, file permissions, uinput, or shutdown behavior.
3. Check `CONFIGURATION_REFERENCE.md` before changing a file format, default, installed path, port, controller mapping, or service unit.
4. Never commit `data/`, tokens, passwords, pairing codes, the downloaded hand model, caches, or locally generated App Lab ZIP files.

Security vulnerabilities should follow the private reporting process in `SECURITY.md`, not an ordinary pull request or public issue.

Source style and documentation

Every executable, source file, service unit, and comment-capable application configuration must begin with the standard project header. Preserve a shebang as the first line when one is required. The header must identify:

- project and repository-relative filename;
- concise purpose;
- author and copyright;
- `SPDX-License-Identifier: MIT`;
- a short dated change log;
- `docs/CHANGELOG.md` and Git as the complete history.

Python modules need a useful module docstring. Public classes and functions, plus non-obvious security, protocol, lifecycle, and numerical helpers, need focused docstrings describing their behavior and safety conditions. Comments should explain decisions and constraints rather than repeat the next line of code.

Use four-space Python indentation, type annotations for interfaces, descriptive names, bounded network and file operations, and existing project patterns. Shell scripts use the shell declared by their shebang; Bash scripts should keep `set -euo pipefail`. Avoid adding a dependency when the standard library or an existing dependency handles the requirement clearly.

JSON does not accept comments. Keep JSON files valid and document their fields, consumer, installed location, defaults, and secret status in `CONFIGURATION_REFERENCE.md`.

Run the source audit before committing:

```
SH
scripts/check-source-docs.py
```

Tests and checks

Core tests must remain independent of a physical camera, UNO Q, and RetroPie:

```
SH
PYTHONPATH=src python3 -m unittest discover -s tests -v
python3 -m compileall -q python scripts src tests
```

Run the documentation and syntax checks:

```
SH
scripts/check-documentation.py
for file in scripts/*.sh; do bash -n "$file"; done
for file in retropie/bin/* retropie/*.sh; do sh -n "$file"; done
```

Add or update tests when behavior, validation, security boundaries, packet formats, gesture mappings, configuration parsing, or recovery paths change. Do not add tests that only repeat static configuration without protecting a meaningful contract.

Documentation changes

Project Markdown belongs under `docs/`, except the repository-root `README.md`. Update all affected guides when a user-visible command, path, control, screen, configuration field, dependency, or troubleshooting procedure changes.

Add user-visible work to the appropriate category under `Unreleased` in `docs/CHANGELOG.md`. Git history remains the exact implementation record; do not paste a full Git log into individual source headers.

Rebuild and inspect all PDF editions after changing maintained Markdown:

```
SH
python3 scripts/build-docs-pdf.py
scripts/check-documentation.py --require-pdfs
```

Commit the updated Markdown and PDFs together.

Public package changes

Build and verify the model-free App Lab package with:

```
SH
scripts/build-app-lab-package.sh
scripts/verify-app-lab-package.py
```

The ZIP must contain public source, configuration examples, documentation, and the required custom UNO Q MediaPipe wheel. It must exclude private `data/`, the downloaded Google model, tests, PDFs, caches, and Git history. Do not force-add the generated ZIP to Git. GitHub Actions publishes a short-lived verified ZIP artifact for each successful workflow run; tagged releases may attach a verified ZIP for long-term distribution.

Commits and pull requests

- Use a short imperative commit subject that describes the outcome.

- Explain what changed, why it was needed, and how it was verified.
- Keep unrelated cleanup separate from behavioral changes.
- Call out migration, deployment, compatibility, security, and rollback risks.
- Do not claim hardware verification unless the change was tested on the named device. Automated tests and simulated input should be described accurately.
- Wait for the GitHub Actions quality workflow to pass before merging.